

IoT

ESP32-CAM +

Arduino

گزارش پروژه درس مبانی اینترنت اشیا

عنوان پروژه

طراحی و پیاده سازی سامانه هوشمند اینترنت اشیا برای
کنترل چند رابطهای تجهیزات، دریافت تصویر و فرمان
صوتی/متنی

Intelligent Multi-Interface IoT Control with ESP32-CAM, Arduino, Telegram, Voice, and
LLM Commands

استاد درس: دکتر علی بهلولی

اعضای گروه:

سپهر رجبی - ۴۰۱۳۶۲۳۰۰۸

امیرحسین جعفری - ۴۰۱۳۶۱۳۰۱۵

بهار ۱۴۰۳

پروژه حاضر یک سامانه اینترنت اشیا چن درابطه‌ای را پیاده‌سازی می‌کند که در آن یک ESP32-CAM، یک برد Arduino و یک رایانه میزبان به صورت هم‌زمان در چرخه دریافت فرمان، تفسیر فرمان، کنترل عملکرد و دریافت تصویر مشارکت دارند. ESP32-CAM در نقش دروازه محلی، نقطه دسترسی Wi-Fi و وب‌سرور را ایجاد می‌کند، امکان ورود کاربر، ارسال فرمان متنی و درخواست تصویربرداری را فراهم می‌سازد و دو خروجی نوری را کنترل می‌کند. Arduino نیز دو LED و یک بازر را از طریق پیوند سریال کنترل می‌کند. در لایه رایانه، برنامه پایتون به عنوان هماهنگ‌کننده، ارتباط سریال با هر دو برد، ربات تلگرام، دریافت پیام صوتی، تبدیل گفتار به متن و نگاشت فرمان طبیعی به الفبای کنترلی محدود A-L را مدیریت می‌کند. علاوه بر این، تصویر گرفته‌شده با دوربین در یک قاب باینری مشخص از طریق UART به رایانه انتقال می‌یابد.

در کد جداگانه ارائه‌شده، از کتابخانه DeepFace برای تطبیق یک تصویر پرس‌وجو با چند تصویر مرجع استفاده شده است؛ این ماژول در نسخه اولیه به خط لوله اصلی متصل نیست و بنابراین در این گزارش به عنوان قابلیت مکمل تحلیل می‌شود. تحلیل مهندسی پروژه نشان می‌دهد که ایده مرکزی، یعنی جداکردن «رابط کاربر و استدلال نرم‌افزاری» از «کنترل قطعی سخت‌افزار»، طراحی مناسبی برای یک نمونه آموزشی است. با این حال، نسخه اولیه دارای محدودیت‌هایی از جمله ذخیره مستقیم کلیدهای سرویس در کد، دو منطق موازی برای دریافت تصویر، عدم تطابق رفتار اسکرپت درخواست تصویر با پروتکل واقعی، و ساده‌بودن احراز هویت وب است. نسخه آماده گیت‌هاب همراه این گزارش این موارد را تا حد ممکن بدون تغییر ماهیت پروژه اصلاح و مستندسازی می‌کند.

واژگان کلیدی: اینترنت اشیا، ESP32-CAM، Arduino، UART، تلگرام، فرمان صوتی، مدل زبانی، DeepFace، کنترل عملکرد، انتقال تصویر.

خلاصه معماری: ورودی کاربر از وب محلی یا تلگرام دریافت می‌شود؛ پیام صوتی در صورت نیاز به متن تبدیل می‌شود؛ یک مدل زبانی فقط نقش نگاشت‌کننده زبان طبیعی به حروف کنترلی را دارد؛ سپس برنامه پایتون حروف معتبر را به ESP32-CAM یا Arduino می‌فرستد. مسیر تصویر مستقل است؛ درخواست capture / در وب، تصویربرداری را فعال می‌کند و JPEG با قالب START_IMG + length + payload + END_IMG روی سریال به رایانه باز می‌گردد.

فهرست مطالب

۱	چکیده
۱	مقدمه و تعریف مسئله
۳	۱.۱ مبنای تحلیل گزارش
۳	۲ اهداف عملکردی و سناریوهای کاربری
۳	۳ معماری کلان سامانه
۴	۱.۳ لایه رابط انسانی
۴	۲.۳ لایه هماهنگ‌کننده
۴	۳.۳ لایه فیزیکی
۵	۴ سخت‌افزار و نگاشت پایه‌ها
۵	۵ نرم‌افزار ESP32-CAM و رابط وب
۵	۱.۵ راه‌اندازی دوربین
۵	۲.۵ مسیرهای وب

۶	الفبای فرمان و مسیریابی قطعی
۶	۷ برنامه پایتون به عنوان هماهنگ‌کننده مرکزی
۷	۱.۷ مدیریت پورت‌های سریال
۷	۲.۷ تفسیر زبان طبیعی با مدل زبانی
۷	۳.۷ مدیریت رخداد‌های ورود
۷	۴.۷ هم‌زمانی
۷	۸ تلگرام و فرمان صوتی
۷	۹ پروتکل تصویربرداری و انتقال JPEG روی UART
۸	۱.۹ برآورد زمان انتقال
۸	۲.۹ ناهم‌خوانی اسکریپت Tel_bot.py
۸	۱۰ مازول تشخیص چهره با DeepFace
۹	۱۱ تحلیل نقاط قوت طراحی
۹	۱۲ محدودیت‌ها و اشکالات نسخه اولیه
۹	۱.۱۲ کلیدها و توکن‌های سخت‌کدشده
۹	۲.۱۲ دریافت تصویر دوگانه و مسدودکننده
۹	۳.۱۲ فراخوانی نامعتبر در مسیر باقی‌مانده داده
۱۰	۴.۱۲ احراز هویت ساده
۱۰	۵.۱۲ وابستگی به سرویس‌های بیرونی
۱۰	۱۳ آماده‌سازی نسخه حرفه‌ای برای GitHub
۱۰	۱.۱۳ ساختار مخزن
۱۰	۱۴ راهبرد آزمون و سناریوهای پذیرش
۱۱	۱.۱۴ معیارهای قابل اندازه‌گیری
۱۱	۱۵ تحلیل امنیت، حریم خصوصی و قابلیت اطمینان
۱۱	۱.۱۵ مدل تهدید پایه
۱۱	۲.۱۵ کنترل‌های حداقلی پیشنهادی
۱۲	۳.۱۵ اعتمادپذیری مدل زبانی
۱۲	۱۶ پیشنهاد‌های توسعه فنی
۱۲	۱۷ نتیجه‌گیری
۱۲	ضمیمه: راهنمای اجرای نسخه مخزن

۱ مقدمه و تعریف مسئله

هدف پروژه، ساخت یک نمونه عملی از یک سامانه اینترنت اشیا است که صرفاً یک «کلید روشن/خاموش تحت شبکه» نباشد، بلکه چند نوع رابط ورودی و چند گره سخت‌افزاری را زیر یک منطق هماهنگ‌کننده یکپارچه کند. در این معماری، کاربر می‌تواند از طریق مرورگر متصل به شبکه محلی ESP32، پیام تلگرام یا پیام صوتی فرمان بدهد؛ رایانه فرمان را پردازش و به یک پروتکل بسیار کوچک و قطعی تبدیل می‌کند؛ سپس عملکرد متناظر روی یکی از دو برد فعال می‌شود. هم‌زمان، دوربین ESP32-CAM قابلیت تصویربرداری و انتقال فایل باینری به میزبان را نیز فراهم می‌کند.

از دید اینترنت اشیا، پروژه چهار مؤلفه اصلی را پوشش می‌دهد: **حسگری** از طریق دوربین، **عملگری** با LED و بازر، **ارتباط** با Wi-Fi/HTTP و UART، و **هوشمندی/هماهنگ‌سازی** در لایه نرم‌افزار رایانه. این جداسازی با برداشت کلاسیک از سامانه‌های IoT به عنوان شبکه‌ای از اشیا فیزیکی دارای قابلیت حس، ارتباط و کنش سازگار است [۱].

۱.۱ مبای تحلیل گزارش

این گزارش از روی فایل‌های پیاده‌سازی ارائه‌شده بازسازی شده است: برنامه اصلی پایتون، برنامه کمکی درخواست تصویر، کد Arduino، کد ESP32-CAM و اسکریپت مستقل تشخیص چهره. بنابراین هرچا میان «رفتار مورد انتظار» و «رفتار واقعی کد» اختلاف وجود داشته، رفتار واقعی کد مبنا قرار گرفته و اختلاف به صورت شفاف در بخش محدودیت‌ها توضیح داده شده است.

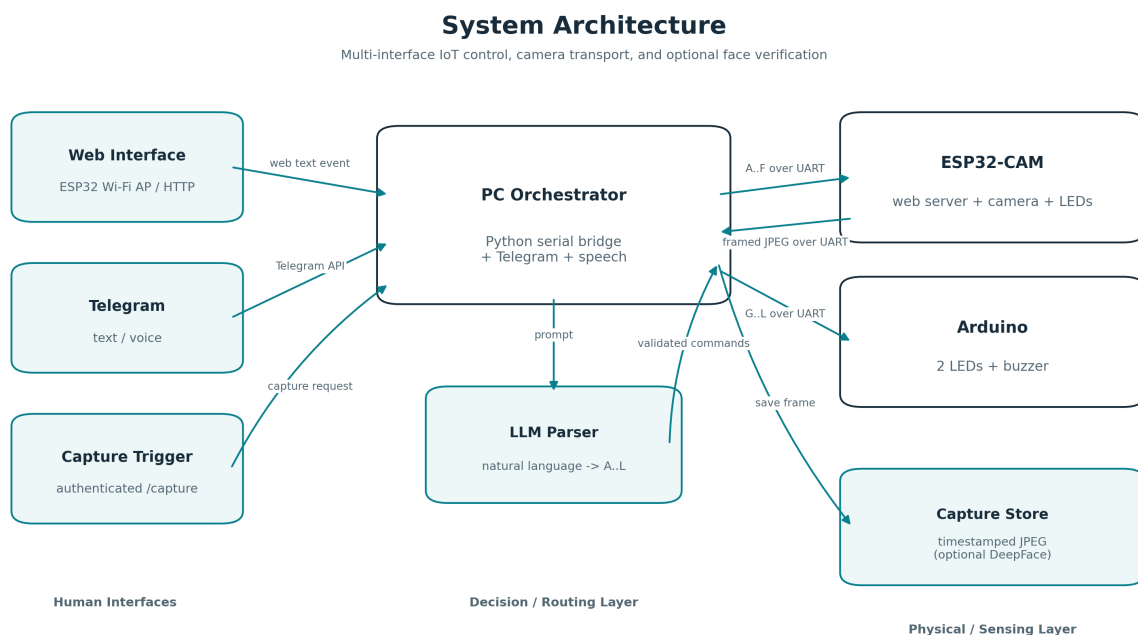
۲ اهداف عملکردی و سناریوهای کاربری

می‌توان نیازمندی‌های پروژه را به شش سناریوی اصلی تقسیم کرد:

۱. **ورود از وب محلی:** کاربر به نقطه دسترسی ESP32 متصل شده، نام کاربری و رمز را وارد می‌کند و در صورت موفقیت به صفحه پیام هدایت می‌شود.
۲. **فرمان متنی از وب:** متن طبیعی کاربر از صفحه وب روی سریال چاپ می‌شود، رایانه آن را دریافت و به یک یا چند حرف کنترلی تبدیل می‌کند.
۳. **فرمان متنی تلگرام:** در صورتی که هنگام ورود گزینه تلگرام فعال شده باشد، ربات تلگرام پیام کاربر را دریافت و از همان خط لوله تفسیر فرمان عبور می‌دهد.
۴. **فرمان صوتی تلگرام:** پیام صوتی دالود، از OGG به WAV تبدیل و با موتور تشخیص گفتار به متن تبدیل می‌شود؛ خروجی متنی سپس مانند سایر فرمان‌ها پردازش می‌شود.
۵. **بازخورد ورود ناموفق:** در نسخه پروژه، پیام خطای ورود از ESP32 به رایانه می‌رسد و رایانه با فرمان‌های K و L بازر آردوینو را برای مدت کوتاه فعال می‌کند.
۶. **تصویربرداری:** کاربر مسیر capture / را فراخوانی می‌کند؛ دوربین فریم JPEG می‌گیرد و تصویر به صورت باینری روی UART به میزبان انتقال داده می‌شود.

۳ معماری کلان سامانه

شکل ۱ توپولوژی منطقی پروژه را نشان می‌دهد. نکته کلیدی این است که مدل زبانی مستقیماً به پایه‌های سخت‌افزار دسترسی ندارد. خروجی مدل ابتدا توسط برنامه میزبان به مجموعه محدود حروف معتبر تقلیل داده شده و سپس بر اساس مقصد به پورت سریال مناسب فرستاده می‌شود. این جداسازی، فضای خروجی را از زبان آزاد به یک پروتکل کوچک تبدیل می‌کند و امکان ممیزی رفتار را افزایش می‌دهد.



شکل ۱: معماری سطح بالای سامانه و مسیرهای اصلی داده و فرمان

۱.۳ لایه رابط انسانی

رابط وب در خود ESP32-CAM میزبانی می‌شود و برای اجرای سناریو به سرویس‌دهنده وب جداگانه نیاز ندارد. تلگرام و پیام صوتی در سمت رایانه مدیریت می‌شوند. از نظر معماری، این دو مسیر در نقطه «متن فرمان» به یکدیگر می‌رسند؛ یعنی پس از تبدیل صوت به متن، تفاوتی میان فرمان وب، تلگرام یا گفتار وجود ندارد.

۲.۳ لایه هماهنگ‌کننده

برنامه پایتون واسط میان همه سرویس‌هاست: دو پورت سریال را باز می‌کند، پیام‌های وضعیت ESP32 را می‌خواند، ربات تلگرام را در یک رشته جداگانه اجرا می‌کند، صوت را تبدیل و تشخیص می‌دهد، مدل زبانی را فراخوانی می‌کند و در نهایت فرمان را مسیریابی می‌کند. از این رو رایانه عملاً نقش edge gateway/orchestrator را دارد.

۳.۳ لایه فیزیکی

ESP32-CAM هم گره حسگر و هم گره عملگر است؛ Arduino گره عملگر دوم است. در طراحی ارائه شده، هر دو برد از طریق اتصال سریال مستقل به رایانه وصل هستند و نرخ تبادل هر دو 115200 baud است.

۴ سخت‌افزار و نگاشت پایه‌ها

جدول ۱: خلاصه اجزای سخت‌افزاری قابل استخراج از کد

گره	پایه/واسط	کارکرد
ESP32-CAM	GPIO 4	خروجی نوری اول؛ در بسیاری از بردهای AI Thinker این پایه با فلش روی برد نیز مرتبط است.
ESP32-CAM	GPIO 33	خروجی نوری دوم؛ منطق موجود در پروژه برای روشن/خاموش آن معکوس است و عملاً سیم‌بندی active-low را نشان می‌دهد.
ESP32-CAM	رابط دوربین	پیکربندی استاندارد پین‌های دوربین نوع AI Thinker و خروجی JPEG.
Arduino	D2	LED 1.
Arduino	D4	LED 2.
Arduino	D3	بازر.
رایانه	دو پورت سریال	یک اتصال مستقل برای ESP32-CAM و یک اتصال برای Arduino.

در کد اولیه، فرمان‌های E و F برای LED 3 تعریف شده‌اند اما پایه مربوطه به دلیل تعارض/محدودیت سخت‌افزاری غیرفعال شده است. بنابراین این دو فرمان در پیاده‌سازی واقعی اثری ندارند. نسخه آماده مخزن، آن‌ها را به صراحت به عنوان **رزرو شده** علامت‌گذاری می‌کند تا قرارداد نرم‌افزار با وضعیت سخت‌افزار اشتباه نشود.

۵ نرم‌افزار ESP32-CAM و رابط وب

کد ESP32-CAM سه وظیفه عمده دارد: راه‌اندازی دوربین، ساخت شبکه Wi-Fi در حالت SoftAP، و ارائه وب‌سرور محلی. در نسخه اولیه، SSID برابر ESP32 بوده و کاربر پس از اتصال معمولاً از نشانی محلی پیش‌فرض 192.168.4.1 استفاده می‌کند.

۱.۵ راه‌اندازی دوربین

دوربین با قالب PIXFORMAT_JPEG، وضوح QVGA معادل 320×240 و یک frame buffer تنظیم شده است. انتخاب وضوح پایین در این پروژه منطقی است، زیرا مسیر انتقال تصویر، UART با نرخ ۱۱۵۲۰۰ باود است و نه یک مسیر پرسرعت شبکه. در کتابخانه رسمی esp32-camera نیز دریافت فریم بر پایه `esp_camera_fb_get()` انجام می‌شود [۲].

۲.۵ مسیرهای وب

سه مسیر اصلی تعریف شده است:

- `/`: صفحه ورود و دریافت وضعیت فعال‌سازی ربات تلگرام؛
- `/message`: صفحه ارسال فرمان طبیعی؛ متن از طریق `Serial.println` به رایانه ارسال می‌شود؛
- `/capture`: ثبت تصویر و ارسال آن روی سریال.

پس از ورود موفق، ESP32 یکی از دو پیام متنی «ورود موفق با تلگرام فعال» یا «ورود موفق» را روی سریال می‌فرستد. برنامه پایتون دقیقاً از همین رشته برای تصمیم درباره راه‌اندازی ربات استفاده می‌کند. این طراحی ساده است و برای آزمایشگاه خوانایی خوبی دارد، اما وابستگی شدید به رشته‌های متنی دقیق ایجاد می‌کند؛ در نسخه صنعتی بهتر است پیام‌های کنترلی ساخت‌یافته باشند.

۶ الفبای فرمان و مسیریابی قطعی

برای جداکردن زبان طبیعی از فرمان سخت‌افزاری، پروژه یک الفبای کوچک از A تا L تعریف کرده است. جدول ۲ نگاشت واقعی این حروف را نشان می‌دهد.

جدول ۲: قرارداد فرمان میان نرم‌افزار میزبان و میکروکنترلرها

فرمان	مقصد	عمل
A	ESP32	روشن کردن خروجی نوری ۱
B	ESP32	خاموش کردن خروجی نوری ۱
C	ESP32	روشن کردن خروجی نوری ۲، با منطق active-low
D	ESP32	خاموش کردن خروجی نوری ۲
E/F	ESP32	رزرو؛ خروجی سوم در سخت‌افزار ارائه شده متصل نیست
G/H	Arduino	روشن/خاموش کردن LED 1
I/J	Arduino	روشن/خاموش کردن LED 2
K/L	Arduino	روشن/خاموش کردن بازر

اگر x متن طبیعی ورودی و $f(x)$ خروجی مدل زبانی باشد، نسخه امن‌تر منطق پروژه را می‌توان به صورت زیر نوشت:

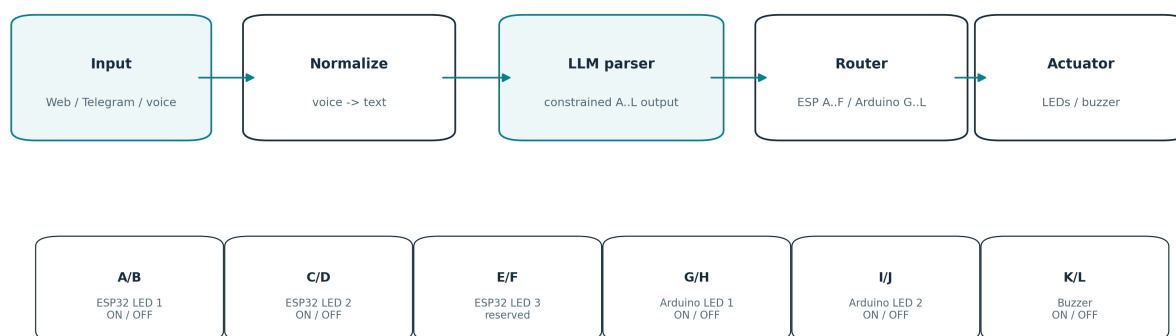
$$V = \{A, B, C, D, E, F, G, H, I, J, K, L\}, \quad \hat{C} = \{c \in f(x) \mid c \in V\}.$$

سپس تابع مسیریابی g تنها بر اساس عضویت در دو زیرمجموعه عمل می‌کند:

$$g(c) = \begin{cases} \text{ESP32}, & c \in \{A, \dots, F\}, \\ \text{Arduino}, & c \in \{G, \dots, L\}. \end{cases}$$

این طراحی از نظر مهندسی مهم است: مدل زبانی «درایور سخت‌افزار» نیست؛ فقط مفسر ورودی طبیعی است و خروجی آن باید قبل از اعمال روی سخت‌افزار اعتبارسنجی شود.

Natural-Language Command Pipeline



Only validated single-letter commands are forwarded to hardware.

شکل ۲: خط لوله تبدیل فرمان طبیعی به پروتکل محدود و سپس مسیریابی به گره مناسب

۷ برنامه پایتون به عنوان هماهنگ‌کننده مرکزی

نسخه اولیه IoT.py مجموعه بزرگی از مسئولیت‌ها را در یک فایل قرار داده است. با وجود تراکم، جریان اصلی آن را می‌توان به چهار زیرسامانه تفکیک کرد.

۱.۷ مدیریت پورتهای سریال

دو پورت مستقل با نرخ یکسان باز می‌شوند. برای ESP32، سیگنال‌های DTR/RTS غیرفعال شده‌اند تا احتمال ریست ناخواسته برد هنگام بازکردن پورت کاهش یابد. برنامه ابتدا منتظر پیام CONFIG_COMPLETE می‌ماند و سپس وضعیت ورود را دنبال می‌کند.

۲.۷ تفسیر زبان طبیعی با مدل زبانی

در پرامپت سیستمی، معنای هر حرف A-L به مدل داده می‌شود و از مدل خواسته می‌شود صرفاً حروف معتبر، جداشده با فاصله، برگرداند. این الگو برای پروژه آموزشی مناسب است، زیرا به جای تولید کد یا فراخوانی تابع آزاد، یک خروجی کوچک و قابل کنترل تولید می‌شود. نسخه آماده گیت‌هاب یک مرحله اعتبارسنجی صریح نیز اضافه کرده است تا هر توکن خارج از مجموعه مجاز حذف شود. استفاده از رابط ChatOpenAI از بسته LangChain در این نقش با الگوی رایج فراخوانی یک مدل سازگار با OpenAI API هم‌راستا است [۴].

۳.۷ مدیریت رخدادهای ورود

سه حالت اصلی از سریال تشخیص داده می‌شود: آماده‌شدن پیکربندی، ورود موفق، و ورود ناموفق. در حالت ورود ناموفق، برنامه K و سپس L را برای آردوینو می‌فرستد و یک بوق کوتاه ایجاد می‌کند. این یک نمونه مناسب از پیوند میان رخداد نرم‌افزاری در گره شبکه و بازخورد فیزیکی در گره دیگر است.

۴.۷ هم‌زمانی

ربات تلگرام فقط در صورت فعال‌شدن گزینه مربوطه راه‌اندازی می‌شود و در یک رشته جداگانه اجرا می‌گردد تا حلقه سریال متوقف نشود. این انتخاب از نظر معماری صحیح است، هرچند در نسخه عمومی باید مراقب مدیریت event loop و سیگنال‌ها در رشته فرعی بود. نسخه آماده مخزن این بخش را با حلقه رویداد مستقل و غیرفعال‌کردن سیگنال‌های توقف در رشته ربات پایدارتر می‌کند.

۸ تلگرام و فرمان صوتی

در پروژه از چارچوب python-telegram-bot برای دریافت پیام‌ها استفاده شده است؛ ساخت Application و افزودن MessageHandler با الگوی نسخه‌های جدید این کتابخانه سازگار است [۳].

برای پیام صوتی، فرایند زیر اجرا می‌شود:

۱. فایل صوتی تلگرام بر اساس شناسه فایل دانلود می‌شود؛
۲. فایل OGG با کتابخانه soundfile به WAV تبدیل می‌شود؛
۳. SpeechRecognition فایل WAV را به سرویس تشخیص گفتار می‌فرستد؛
۴. متن حاصل به کاربر بازگردانده و سپس، در صورت فعال‌بودن کنترل تلگرام، به مدل زبانی داده می‌شود؛
۵. فایل‌های موقت صوتی پاک می‌شوند.

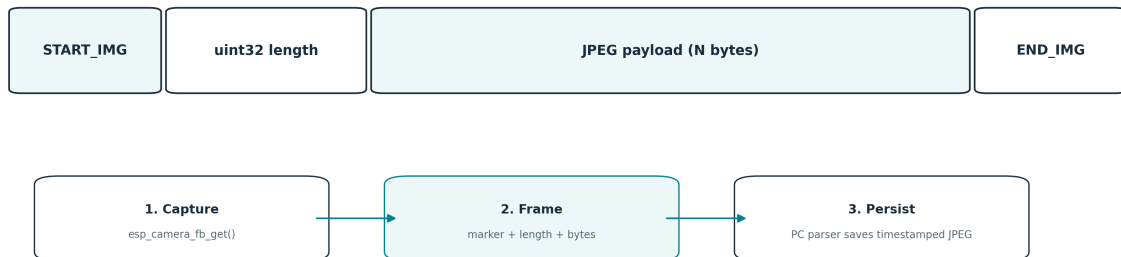
مزیت مهم این معماری آن است که مسیر صوت پس از مرحله رونویسی، هیچ منطق کنترلی ویژه‌ای ندارد و از همان مسیر متن استفاده می‌کند. در نتیجه نگهداری منطق فرمان فقط در یک نقطه انجام می‌شود.

۹ پروتکل تصویربرداری و انتقال JPEG روی UART

در ESP32-CAM، مسیر /capture با esp_camera_fb_get() یک فریم می‌گیرد و سپس به جای برگرداندن تصویر در پاسخ HTTP، آن را روی سریال می‌فرستد. قاب ارسالی شامل یک نشانگر آغاز، طول تصویر، خود بایت‌های JPEG و نشانگر پایان است.

ESP32-CAM UART Image Framing

A fixed marker + 4-byte little-endian length lets the PC separate binary JPEG data from text events.



شکل ۳: قاب انتقال تصویر بین ESP۳۲-CAM و برنامه رایانه

در نسخه اولیه، سمت پایتون چهار بایت برای طول در نظر می‌گیرد و سمت ESP32 مقدار size_t را با sizeof(length) می‌فرستد. روی ESP32 این مقدار عملاً چهار بایت است، اما قرارداد پروتکل بهتر است وابسته به اندازه نوع داده کامپایلر نباشد. نسخه آماده مخزن، طول را صریحاً به uint32_t تبدیل می‌کند.

۱.۹ برآورد زمان انتقال

در UART 8N1 تقریباً برای هر بایت داده ۱۰ بیت روی سیم منتقل می‌شود. بنابراین زمان ایده‌آل انتقال یک تصویر N بایتی در نرخ ۱۱۵۲۰۰ باود تقریباً برابر است با:

$$T_{UART}(N) \approx \frac{10N}{115200} \text{ s.}$$

برای مثال اگر یک فریم ۲۰ هزار بایت باشد، فقط زمان انتقال خام حدود ۱.۷۴ ثانیه خواهد بود. این محاسبه توضیح می‌دهد چرا پروژه وضوح QVGA و کیفیت فشرده‌سازی نسبتاً پایین‌تر را انتخاب کرده است.

۲.۹ ناهم‌خوانی اسکریپت Tel_Iot.py

یک نکته مهم در تحلیل کد این است که اسکریپت Tel_Iot.py پس از ورود، مسیر capture را با HTTP فراخوانی و پاسخ را مستقیماً در capture.jpg ذخیره می‌کند. اما پیاده‌سازی ESP32 در این مسیر یک صفحه HTML با پیام «تصویر از UART ارسال شد» برمی‌گرداند؛ خود تصویر هرگز بدنه پاسخ HTTP نیست. بنابراین فایل ذخیره‌شده توسط اسکریپت اولیه، تصویر واقعی نیست. ابزار tools/capture_request.py در نسخه مخزن این مشکل را اصلاح می‌کند: مسیر وب را فقط برای فعال‌کردن تصویربرداری فراخوانی می‌کند و ذخیره JPEG را به دریافت‌کننده سرپال واگذار می‌کند.

۱۰ مازول تشخیص چهره با DeepFace

فایل جداگانه Face_Rec.py از DeepFace.verify برای مقایسه یک تصویر پرس‌وجو با دو تصویر مرجع استفاده می‌کند. منطق آن ساده و روشن است: ابتدا مسیرهای 1.jpg و 2.jpg به عنوان پایگاه مرجع تعریف شده، 3.jpg به عنوان تصویر مقایسه انتخاب می‌شود و حلقه تا نخستین نتیجه verified=True ادامه می‌یابد. DeepFace یک چارچوب پایتونی برای یکپارچه‌سازی مدل‌ها و عملیات متداول تحلیل چهره است [۵].

مرزبندی مهم: در کدهای ارائه‌شده، این اسکریپت به تابع دریافت تصویر از ESP32-CAM متصل نشده است. بنابراین نمی‌توان گفت سامانه اصلی «پس از هر عکس به صورت خودکار احراز چهره می‌کند». گزارش فقط می‌تواند نتیجه بگیرد که یک نمونه مستقل برای تطبیق چهره نیز در کنار پروژه وجود داشته است. نسخه گیت‌هاب آن را به ابزار خط فرمانی تبدیل می‌کند که مسیر تصویر پرس‌وجو و پوشه تصاویر مرجع را قابل تنظیم می‌سازد.

از منظر امنیت و اخلاق، تصویر چهره داده زیست‌سنجی حساس است. در مخزن عمومی، پوشه `data/reference_faces` عمده‌ای از گیت حذف می‌شود و استفاده از این مازول نباید بدون رضایت، حفاظت داده و ارزیابی خطای واقعی به عنوان مرز اصلی کنترل دسترسی تلقی شود.

۱۱ تحلیل نقاط قوت طراحی

- **تفکیک زبان طبیعی از فرمان سخت‌افزاری:** تبدیل همه درخواست‌ها به یک الفبای محدود باعث می‌شود بخش سخت‌افزار مستقل از شکل جمله کاربر باشد.
- **چندمسیره‌بودن رابط:** وب محلی، تلگرام و صوت بدون تکرار منطق عملگر به یک مسیر مشترک می‌رسند.
- **تقسیم وظایف بین بردها:** ESP32-CAM وظایف شبکه/دوربین را بر عهده دارد و Arduino عملگرهای ساده را کنترل می‌کند؛ این معماری برای نمایش مفهوم گره‌های ناهمگون در IoT آموزشی است.
- **انتقال صریح تصویر:** استفاده از نشانگر و طول فریم، نسبت به ارسال بایت خام بدون قاب‌بندی قابلیت بازسازی بهتری دارد.
- **بازخورد فیزیکی خطای ورود:** ارتباط رخداد امنیتی وب با باز روی گره دیگر نشان‌دهنده یکپارچگی واقعی اجزاست.
- **قابلیت توسعه:** پروتکل حروفی می‌تواند به رله، موتور، حسگر یا گره‌های بیشتر گسترش یابد؛ البته در مقیاس بزرگ بهتر است پیام ساخت‌یافته جایگزین شود.

۱۲ محدودیت‌ها و اشکالات نسخه اولیه

بررسی دقیق کد اولیه چند مورد را نشان می‌دهد که در یک پروژه دانشگاهی طبیعی‌اند، ولی برای مخزن عمومی یا توسعه بعدی باید اصلاح شوند.

۱.۱۲ کلیدها و توکن‌های سخت‌کدشده

در فایل پایتون اولیه، کلید سرویس مدل زبانی و توکن ربات تلگرام مستقیماً داخل کد قرار داده شده‌اند. این مهم‌ترین مشکل انتشار عمومی است. نسخه گیت‌هاب هیچ مقدار واقعی را نگه نمی‌دارد و تنظیمات را از `.env` می‌خواند. همچنین اطلاعات شبکه و ورود ESP32 به فایل `secrets.h` منتقل شده که در `.gitignore` قرار دارد.

اگر کلیدها یا توکن‌های نسخه اولیه قبلاً در هر مخزن، چت اشتراکی یا فایل عمومی قرار گرفته‌اند، فقط حذف آن‌ها از نسخه جدید کافی نیست؛ باید در سرویس مربوطه **باطل و مجدداً صادر** شوند.

۲.۱۲ دریافت تصویر دوگانه و مسدودکننده

در ابتدای `IoT.py` تابعی برای انتظار و دریافت تصویر بلافاصله فراخوانی می‌شود؛ این فراخوانی می‌تواند اجرای سایر بخش‌ها را تا زمان رسیدن عکس متوقف کند. پایین‌تر همان فایل، منطق دوم و پیچیده‌تری برای دریافت تصویر در حلقه اصلی وجود دارد. وجود دو مسیر موازی هم احتمال خطا را بالا می‌برد و هم مشخص نیست کدام مسیر منبع حقیقت است. نسخه آماده مخزن یک **parser افزایشی واحد** برای جریان مخلوط متن/باینری دارد.

۳.۱۲ فراخوانی نامعتبر در مسیر باقی‌مانده داده

در منطق اولیه دریافت تصویر، عبارتی شبیه `self.process_serial_data(...)` در یک تابع آزاد وجود دارد؛ در آن محدوده `self` تعریف نشده است. همچنین بافر پیش از محاسبه داده باقی‌مانده پاک می‌شود. این شاخه در بسیاری از اجراها ممکن است فعال نشود، اما از نظر ایستای کد یک خطای روشن است.

۴.۱۲ احراز هویت ساده

در نسخه اولیه نام کاربری و رمز هر دو مقدار ثابت و بسیار ساده دارند، اطلاعات با HTTP محلی منتقل می‌شود و متغیر `isAuthenticated` یک پرچم سراسری برای کل دستگاه است نه نشست مستقل هر مرورگر. این سطح برای دمو قابل درک است، ولی برای محیط واقعی مناسب نیست.

۵.۱۲ وابستگی به سرویس‌های بیرونی

فرمان تلگرام، تشخیص گفتار و نگاشت مدل زبانی به اتصال اینترنت و سرویس‌های شخص ثالث وابسته است. در صورت قطع اینترنت، مسیر وب محلی همچنان می‌تواند متن را به رایانه برساند، اما اگر تفسیر متن کاملاً به مدل بیرونی وابسته باشد، کنترل طبیعی از کار می‌افتد. یک توسعه مناسب، افزودن `parser` قطعی محلی برای فرمان‌های رایج است.

۱۳ آماده‌سازی نسخه حرفه‌ای برای GitHub

نسخه همراه گزارش با هدف «انتشارپذیری» و نه بازنویسی ماهیت پروژه تهیه شده است. تغییرات اصلی عبارت‌اند از:

۱. ایجاد ساختار پوشه‌ای روشن شامل `src`، `firmware`، `tools`، `docs` و `data`؛
۲. انتقال تنظیمات حساس و وابسته به سیستم به `env.example` و `secrets.h.example`؛
۳. افزودن `gitignore`. برای جلوگیری از ثبت کلیدها، تصویرها، صوت‌ها و داده زیست‌سنجی؛
۴. بازآرایی برنامه میزبان به کلاس هماهنگ‌کننده و یک `parser` واحد برای متن و تصویر روی سرپال؛
۵. اصلاح قرارداد طول تصویر به `uint32`؛
۶. اصلاح اسکریپت درخواست تصویر؛
۷. تبدیل اسکریپت `DeepFace` به ابزار قابل تنظیم بدون مسیر سخت‌کدشده ویندوز؛
۸. مستندسازی کامل روش نصب، فلش‌کردن دو برد، راه‌اندازی و محدودیت‌های امنیتی؛
۹. افزودن سه نمودار معماری، جریان فرمان و پروتکل تصویر.

۱.۱۳ ساختار مخزن

```
IoT-Final-Project/
|-- README.md
|-- SECURITY.md
|-- .env.example
|-- requirements.txt
|-- src/iot_controller.py
|-- src/face_verify.py
|-- tools/capture_request.py
|-- firmware/arduino/Arduino_IOT.ino
|-- firmware/esp32_cam/ESP_AC_IOT.ino
|-- firmware/esp32_cam/secrets.h.example
|-- docs/report.tex
^-- docs/figures/...
```

۱۴ راهبرد آزمون و سناریوهای پذیرش

برای ارزیابی پروژه، آزمون صرفاً «روشن‌شدن یک LED کافی نیست؛ باید زنجیره کامل از رابط تا عملکرد و نیز مسیر تصویر آزموده شود. جدول زیر مجموعه‌ای از آزمون‌های پذیرش پیشنهادی را ارائه می‌کند.

جدول ۳: ماتریس پیشنهادی آزمون سامانه

ردیف	آزمون	نتیجه مورد انتظار
۱	روشن شدن و آماده سازی ESP۳۲	پیام CONFIG_COMPLETE در رایانه دریافت شود.
۲	ورود معتبر بدون تلگرام	صفحه پیام باز شود و ربات راه اندازی نشود.
۳	ورود معتبر با تلگرام	پیام فعال سازی روی سریال دریافت و ربات در رشته جداگانه شروع شود.
۴	ورود نامعتبر	پیام خطا دریافت و بازر آردوینو حدود نیم ثانیه فعال شود.
۵	فرمان وب برای ESP۳۲	مدل حرف A-F تولید کند و فقط پورت ESP۳۲ فرمان را دریافت کند.
۶	فرمان وب برای Arduino	مدل حرف G-L تولید کند و فقط پورت Arduino فرمان را دریافت کند.
۷	پیام تلگرام متنی	همان مسیر اعتبارسنجی و مسیریابی فرمان وب طی شود.
۸	پیام تلگرام صوتی	فایل موقت ایجاد، رونویسی، حذف و فرمان اجرا شود.
۹	تصویربرداری	فریم با نشانگر و طول صحیح دریافت و JPEG با نام زمان دار ذخیره شود.
۱۰	خروجی نامعتبر مدل	هیچ بایت نامعتبر به هیچ میکروکنترلری ارسال نشود.
۱۱	فرمان E/F	رفتار به عنوان رزرو/بدون عمل فیزیکی مستند باشد.
۱۲	تشخیص چهره مستقل	تصویر پرس و جو با مجموعه مرجع مقایسه و نتیجه تطبیق/عدم تطبیق گزارش شود.

۱.۱۴ معیارهای قابل اندازه گیری

برای توسعه بعدی می توان زمان انتهابه انتها از دریافت پیام تا تغییر خروجی، درصد فرمان های مدل که پس از اعتبارسنجی معتبر می مانند، نرخ خطای رونویسی صوت، زمان انتقال تصویر و نرخ شکست فریم را ثبت کرد. برای بخش تشخیص چهره نیز ارزیابی واقعی باید با داده آزمون مستقل و معیارهایی مانند FAR/FRR انجام شود، نه فقط چند تصویر نمونه.

۱۵ تحلیل امنیت، حریم خصوصی و قابلیت اطمینان

۱.۱۵ مدل تهدید پایه

دارایی های مهم سامانه شامل کلیدهای سرویس، توکن تلگرام، اعتبارنامه وب، فرمان های سخت افزار، تصویر دوربین و تصاویر مرجع چهره هستند. مهاجم بالقوه می تواند کاربر غیرمجاز در محدوده Wi-Fi، کاربر تلگرام دارای دسترسی به ربات، یا فردی با دسترسی به مخزن کد باشد.

۲.۱۵ کنترل های حداقلی پیشنهادی

- چرخش کلیدهای قبلی و نگهداری اسرار خارج از گیت؛
- رمز قوی برای Wi-Fi AP و حساب وب؛
- محدود کردن شناسه کاربران/چت های مجاز تلگرام؛
- اعتبارسنجی نهایی هر فرمان پیش از ارسال؛
- افزودن تأیید دریافت و وضعیت فعلی عملگر برای فرمان های مهم؛
- محدود سازی نرخ درخواست های وب و تلگرام؛
- عدم ثبت تصاویر چهره و تصاویر دوربین در مخزن عمومی؛
- نگهداری لاگ رخدادها بدون ذخیره محتوای حساس بیش از نیاز.

۳.۱۵ اعتمادپذیری مدل زبانی

مدل زبانی ممکن است حتی با دستور صریح، خروجی غیرمنتظره تولید کند. به همین دلیل constraint پرامپت به تنهایی کنترل امنیتی کافی نیست. کنترل واقعی در سمت برنامه میزبان است: تنها حروف عضو مجموعه مجاز اجازه عبور دارند. در کاربرد حساس‌تر، حتی این روش نیز باید با سیاست مجوز کاربر، ماشین حالت و محدودیت عملیات همراه شود.

۱۶ پیشنهاد‌های توسعه فنی

۱. **پروتکل پیام ساخت‌یافته:** جایگزینی رشته‌ها و تک‌حرف‌ها با فریم دارای نوع پیام، شناسه، طول، checksum/CRC و ACK. برای پروژه بزرگ‌تر، MQTT نیز گزینه طبیعی است.
۲. **مسیر مستقیم تصویر روی شبکه:** در صورت نیاز به نرخ بالاتر، انتقال JPEG با HTTP یا استریم MJPEG به جای UART می‌تواند گلوگاه را کاهش دهد؛ البته این تغییر معماری فعلی را ساده‌تر ولی متفاوت می‌کند.
۳. **فرمان محلی قطعی:** برای عبارات رایج مانند «چراغ اول را روشن کن» یک parser واژه‌نامه‌ای محلی پیش از مدل زبانی قرار گیرد؛ مدل فقط برای زبان آزاد یا موارد مبهم استفاده شود.
۴. **ماشین حالت تجهیزات:** وضعیت هر خروجی در نرم‌افزار نگهداری و پاسخ سخت‌افزار تأیید شود تا دستور تکراری یا شکست ارتباط قابل تشخیص باشد.
۵. **یکپارچه‌سازی اختیاری تشخیص چهره:** پس از دریافت موفق تصویر، یک hook قابل فعال‌سازی می‌تواند DeepFace را اجرا کند؛ باین حال برای کنترل دسترسی واقعی باید تشخیص زنده‌بودن و سیاست حریم خصوصی نیز اضافه شود.
۶. **آزمون خودکار:** parser سریال را می‌توان بدون سخت‌افزار با تزریق های chunk مصنوعی آزمود؛ همچنین مسیربای فرمان با mock پورت سریال قابل تست واحد است.
۷. **داشبورد وضعیت:** نمایش آخرین فرمان، وضعیت اتصال دو برد، آخرین تصویر و خطاها در یک رابط واحد، قابلیت مشاهده‌پذیری را بالا می‌برد.

۱۷ نتیجه‌گیری

پروژه انجام‌شده یک نمونه نسبتاً کامل از یک سامانه اینترنت اشیا آموزشی است که چند مفهوم مهم را در یک سناریوی واحد کنار هم قرار می‌دهد: شبکه محلی و وب‌سرور روی ESP32-CAM، کنترل عملکرد روی دو میکروکنترلر، ارتباط سریال دوطرفه، دریافت و قاب‌بندی تصویر، ربات تلگرام، فرمان صوتی و استفاده محدودشده از مدل زبانی برای تفسیر زبان طبیعی. ارزش اصلی معماری در این است که رابط‌های متنوع در نهایت به یک قرارداد کنترلی کوچک تبدیل می‌شوند و لایه فیزیکی از جزئیات زبان کاربر مستقل باقی می‌ماند.

تحلیل کد نشان داد که نسخه اولیه در سطح نمونه درسی قابل فهم است، اما برای انتشار عمومی باید اسرار از کد خارج شوند، مسیر دریافت تصویر یکپارچه شود، اختلاف اسکریپت HTTP با انتقال واقعی UART اصلاح شود و محدودیت فرمان‌های رزرو مستند گردد. بسته آماده گیت‌هاب همراه این گزارش این اصلاحات را اعمال می‌کند و در عین حال معماری اصلی پروژه را حفظ می‌کند. اسکریپت DeepFace نیز به عنوان ماژول مکمل و مستقل مستندسازی شده است، نه قابلیتی که به اشتباه بخشی از مسیر اصلی معرفی شود.

از منظر آموزشی، پروژه نشان می‌دهد که IoT فقط اتصال یک برد به اینترنت نیست؛ مسئله اصلی طراحی قرارداد بین گره‌ها، کنترل جریان داده، مدیریت ناهمگونی رابط‌ها، تفکیک مسئولیت‌ها و در نظر گرفتن امنیت و قابلیت نگهداری است. با افزودن پروتکل قابل اطمینان‌تر، احراز هویت قوی‌تر، آزمون خودکار و انتقال شبکه‌ای تصویر، همین نمونه می‌تواند مبنای یک سامانه پیشرفته‌تر قرار گیرد.

ضمیمه: راهنمای اجرای نسخه مخزن

به صورت خلاصه، مراحل اجرای نسخه آماده گیت‌هاب چنین است:

۱. نصب وابستگی‌ها از requirements.txt؛
۲. کپی env.example به env. و تنظیم پورت‌ها، کلید مدل و توکن تلگرام؛
۳. کپی secrets.h.example به secrets.h و تنظیم مشخصات شبکه/وب؛
۴. فلش firmware/esp32_cam/ESP_AC_IOT.ino روی ESP32-CAM؛
۵. فلش firmware/arduino/Arduino_IOT.ino روی Arduino؛
۶. اجرای python src/iot_controller.py؛
۷. اتصال به شبکه ESP32، ورود و آزمایش فرمان‌های وب/تلگرام؛
۸. برای تصویربرداری، استفاده از لینک وب یا python tools/capture_request.py؛
۹. در صورت نیاز به مازول چهره، نصب requirements-face.txt و اجرای src/face_verify.py روی تصویر ذخیره‌شده.

منابع

- [1] Atzori, L., Iera, A., and Morabito, G. (۲۰۱۰) The Internet of Things: A survey. Computer Networks, ۵۴(۱۵), ۲۷۸۷-۲۸۰۵. <https://doi.org/10.1016/j.comnet.2010.05.010>.
- [2] Espressif Systems. esp32-camera: ESP32 compatible camera driver. <https://github.com/espressif/esp32-camera>.
- [3] python-telegram-bot Documentation. <https://docs.python-telegram-bot.org/>.
- [4] LangChain Documentation. ChatOpenAI / OpenAI-compatible chat model integration. <https://python.langchain.com/docs/integrations/chat/openai/>.
- [5] Serengil, S. I., and Ozpinar, A. DeepFace / LightFace face recognition framework and software. <https://github.com/serengil/deepface>.